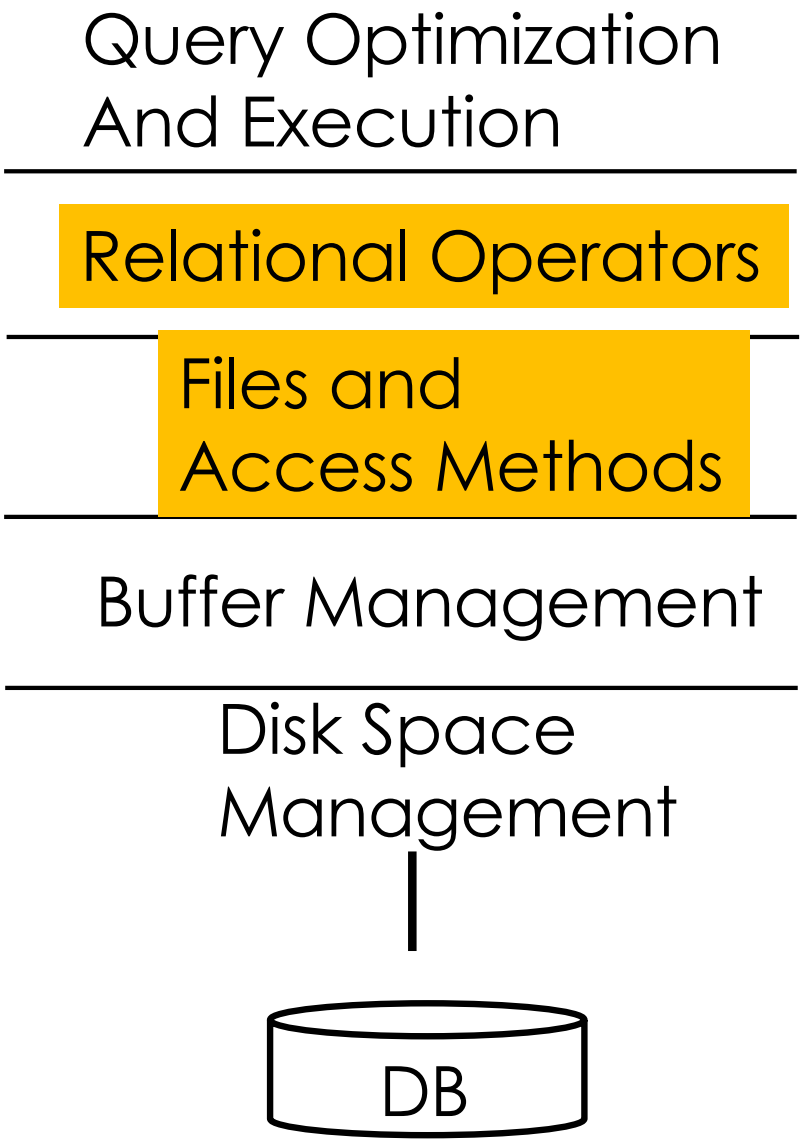
Base operations

directly accessing tables or indexing

Internals of a DBS



Typical basic Operations

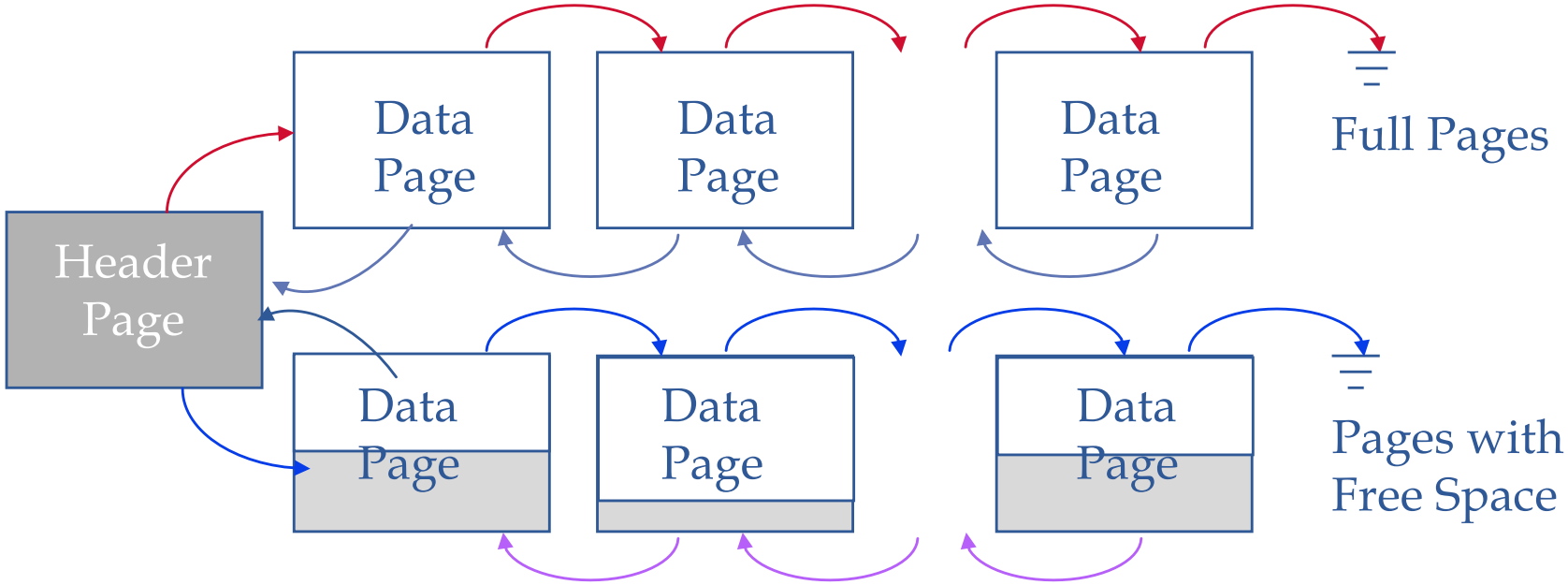
- Scan over all records
  - `SELECT * FROM Students`
- Point Query
  - `SELECT * FROM Students WHERE sid = 100`
- Equality Query
  - `SELECT * FROM Students WHERE starty = 2015`
- Range Search
  - `SELECT * FROM Students`
    `WHERE starty > 2019 and starty <= 2024`
- Insert
  - `INSERT INTO Students VALUES (23, ‘Bertino”,2024,… )`
- Delete
  - `DELETE FROM Students WHERE sid = 100`
  - `DELETE FROM Students WHERE endyear < 1970`
- Update
  - Delete+insert

Reminder of our back-of-the-envelope cost model for execution

- How **real systems** estimate the costs for executing a statement
  - ☆ Number of I/Os
  - ☆ Time for each I/O depending on random/sequential access
  - ☆ CPU Execution Cost
  - ☆ Network Cost in distributed system
- **What we do in this course**
  - ☆ Only consider number of I/Os
  - ☆ Do not consider CPU nor the individual time for each I/O access (ignores random vs. sequential and page pre-fetch)
  - ☆ Only consider read I/O and no writes (read-only workload)
  - ☆ *Average-case analysis*; based on several simplistic assumptions.

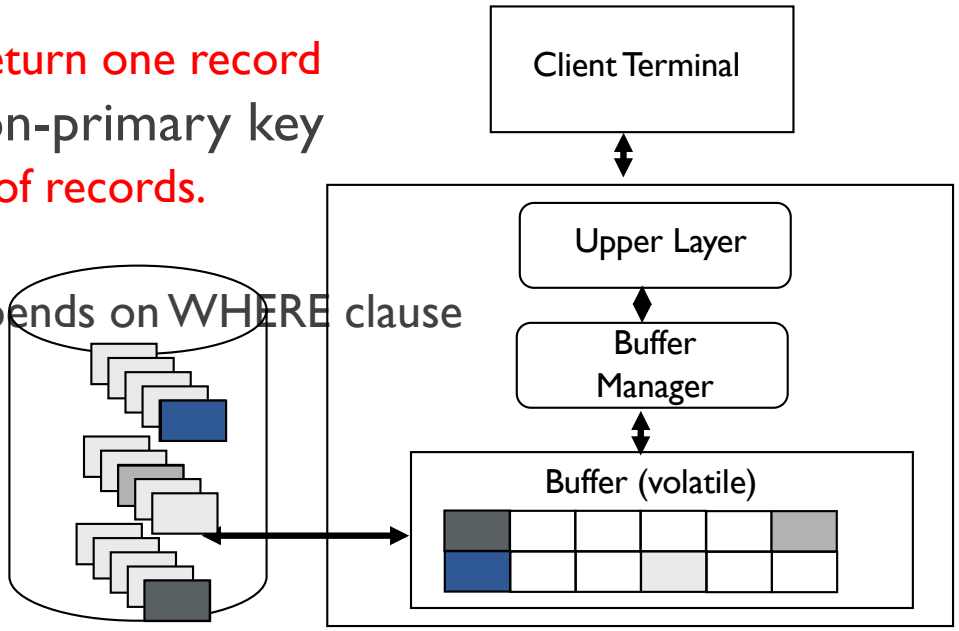
☛ Good enough to show the overall trends!

Unsorted heap file implemented as a list



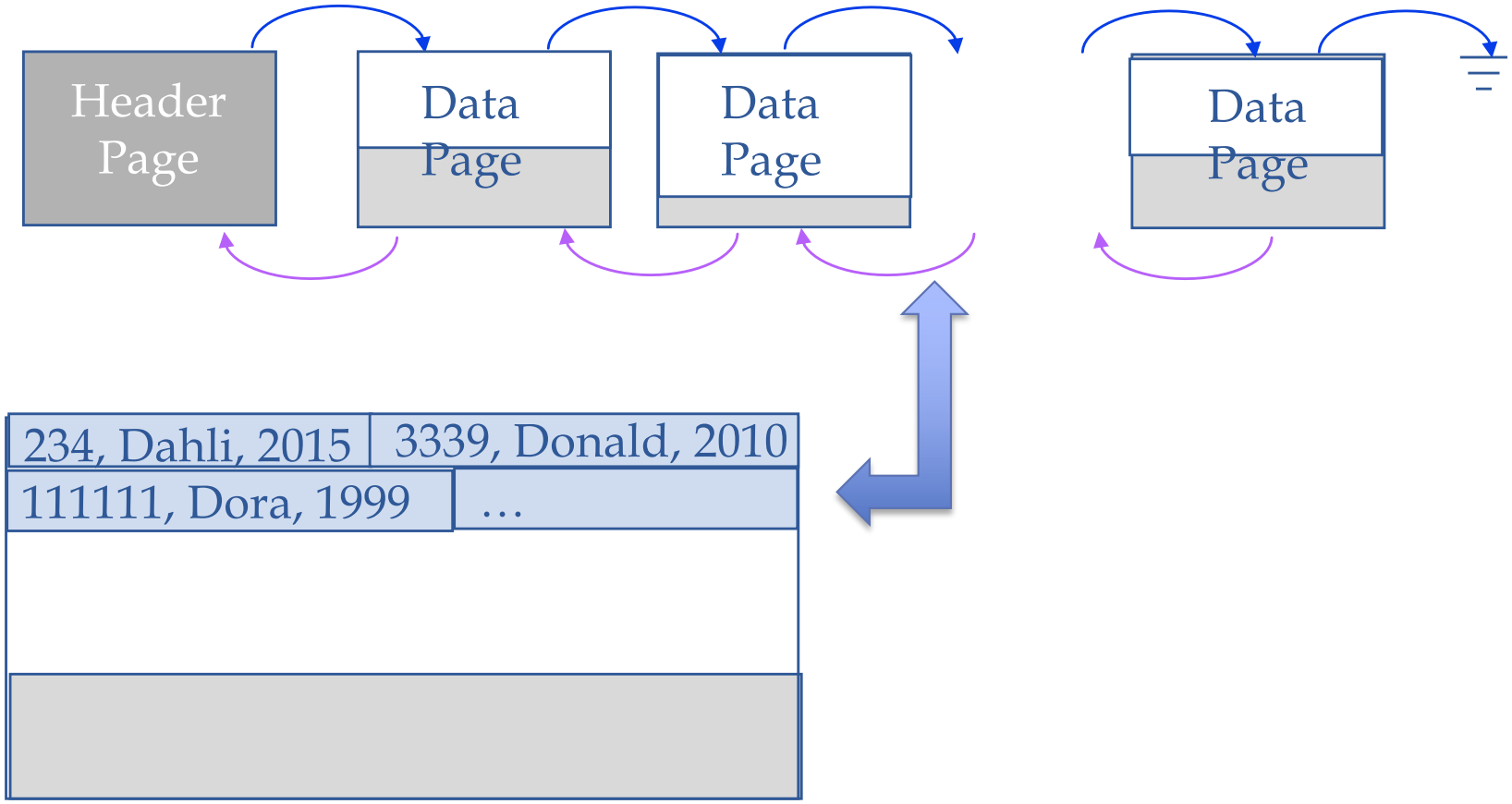
Unsorted Heap File

- How well does it support the different operations?
  - insert
    - Insert in any free page
    - Cost is low (insert anywhere): likely 1 I/O
  - scan retrieving all records (SELECT *)?
    - go from page and page and return each tuple
      - Number I/O = number of data pages for relation
    - Not much optimization possible
    - But tuples can be stored in a compact way
  - Point query on unique attributes
    - Go from page to page, look at each record and return once record is found
    - have to read on avg. half the pages to return one record
  - range search or equality search on non-primary key
    - have to read all pages to return subset of records.
  - delete/update
    - same as for equality/range search -- depends on WHERE clause



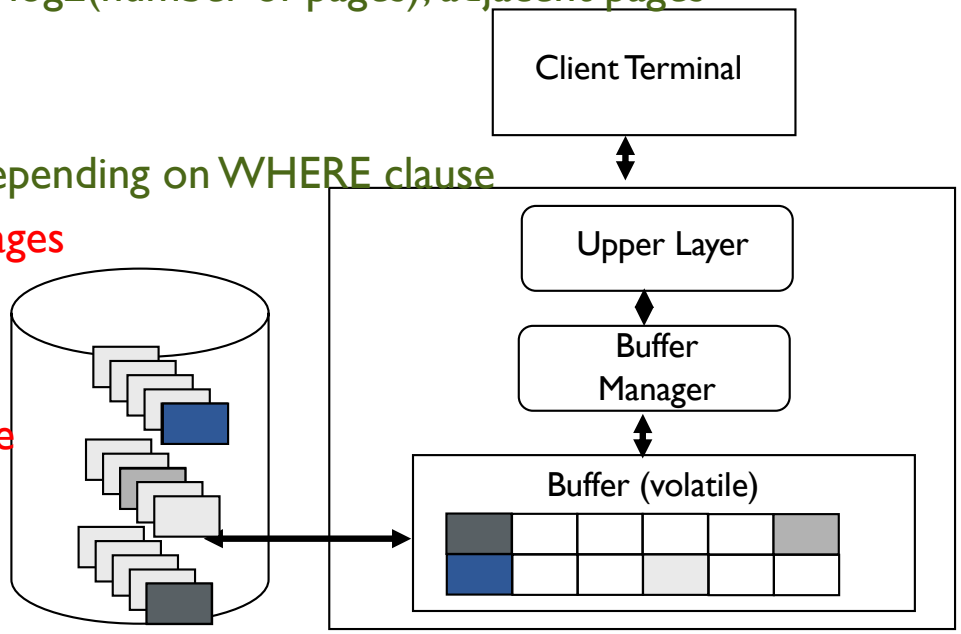
Sorted file

- Records are sorted by one of the attributes (e.g., name).



Sorted File

- insert
  - have to find proper page
  - Algorithm to find proper page? ➔ binary search in log2(number-of-pages)
  - overflow possible
  - less compact that unsorted heap
- scan retrieving all records (SELECT *)?
  - you have to retrieve all pages anyways
- point search on sort attribute
  - find qualifying page with binary search in log2(number-of-pages)
- range search on sort attribute or equality search on non-unique attribute
  - find first qualifying page with binary search in log2(number-of-pages); adjacent pages might have additional matching records
- delete/update
  - finding tuple same as equality/range search depending on WHERE clause
  - update itself might lead to restructuring of pages
- Sorted output: (ORDER BY)
  - good if on sorted attribute
- Search/sort on attributes other than sort attribute
  - Similar to unsorted heap



Estimation Number of Data Pages I

- Our first back of the envelope calculation
- Given / Assumptions
  - About the relation R(A,B,C,D,E,F)
    - A and B are int (each 4 Bytes), C-F are varchar[80]
    - 200,000 tuples
  - Data pages
    - Each data page as 4 KBytes (4000 Bytes) and is around 75% full (=3KB)
    - Ignore whether sorted or not sorted
- 1. calculation: Size of a data record
  - Simplifying assumptions
    - Varchars C-F are on average 40 Bytes
    - Ignore the pointers at the header of variable size records
    - Ignore the possibility of NULL values
  - Average record size is:
    - 4+4+40+40+40+40 = 168

Estimation Number of Data Pages II

- Size of a data record: 168 Bytes
- Number of data records 20000
- Each page has around 4000 * 0.75 = 3000 Bytes of data

- Number of data pages
  - Calculation Alternative 1:
    - Number of tuples on a page
      - 4000 * 0.75 / 168 ≈ 18 tuples in a page on an average.
    - Number of pages:
      - 200,000 tuples in total / 18 per page ≈ 11111 pages in total.
  - Calculation Alternative II
    - Number of tuples * by tuple size / occupied size on page
    - 200,000 * 168 / 3000 = 11200

Indexing

Indexes

- Even a sorted file only supports queries on sorted attributes.
- Solution: Build an index for any attribute (collection of attributes) that is frequently used in queries
  - **Additional information / extra data structure** that helps finding specific tuples faster
    - It's extra / on top of the actual records (that remain in an unsorted heap file or in a sorted file….)
  - We call the collection of attributes over which the index is built the **search key attributes** for the index.
  - Any subset of the attributes of a relation can be the search key for an index on the relation.
  - *Search key* is not the same as *primary key / key candidate*

Creating an index in DB2

- **Simple**
  - `CREATE INDEX ind1 ON Students(sid);`
  - `DROP INDEX ind1;`

- The search key is sid (it could be any other attribute of the relation)
- An index on sid helps with queries that have a equality condition on sid (and sometimes also helpful for range queries)
  - sid = 58
  - sid <10

First Idea

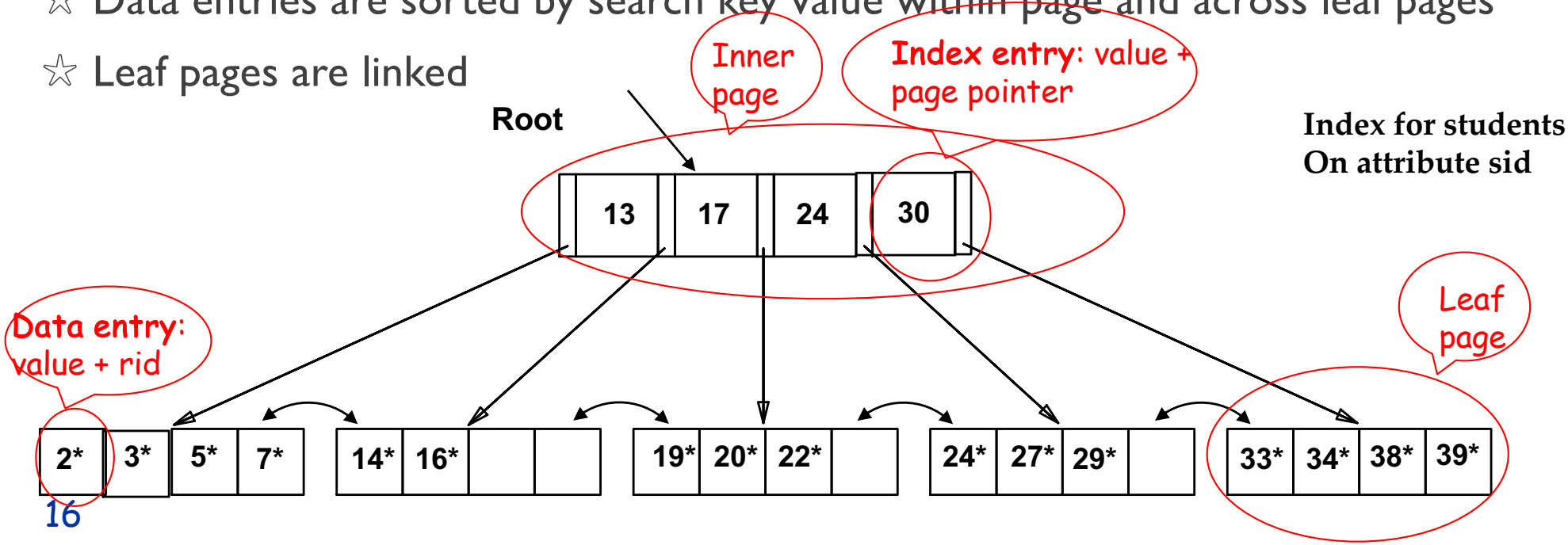
Sid	rid
2	(2,3)
3	(2,7)
5	(4,10)
7	(22,3)
…	

Size of index (rid = 10 Bytes):
2 Mio * 4+10 on 7000 pages if tightly packed

➔ Index is also a file with many pages

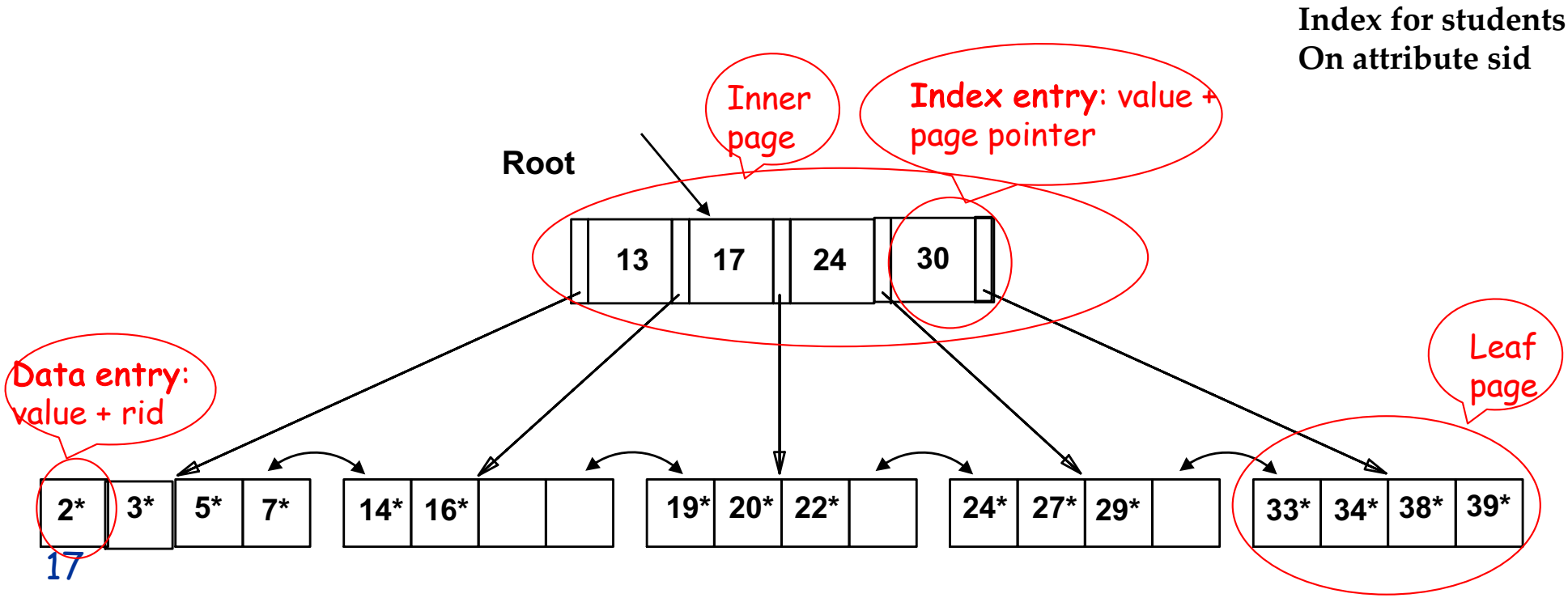
B+ Tree: The Most Widely Used Index

- Each node/leaf represents one page
  - ☆ Since the page is the transfer unit to disk
- Leafs contain *data entries* (denoted as k*)
  - ☆ For now, assume each data entry *refers to* one tuple. The data entry consists of two parts
    - Value of the search key (k)
    - Record identifier (rid = (page-id, slot))
  - ☆ That is: data entry is NOT a tuple but a pointer to a tuple
  - ☆ Data entries are sorted by search key value within page and across leaf pages
  - ☆ Leaf pages are linked



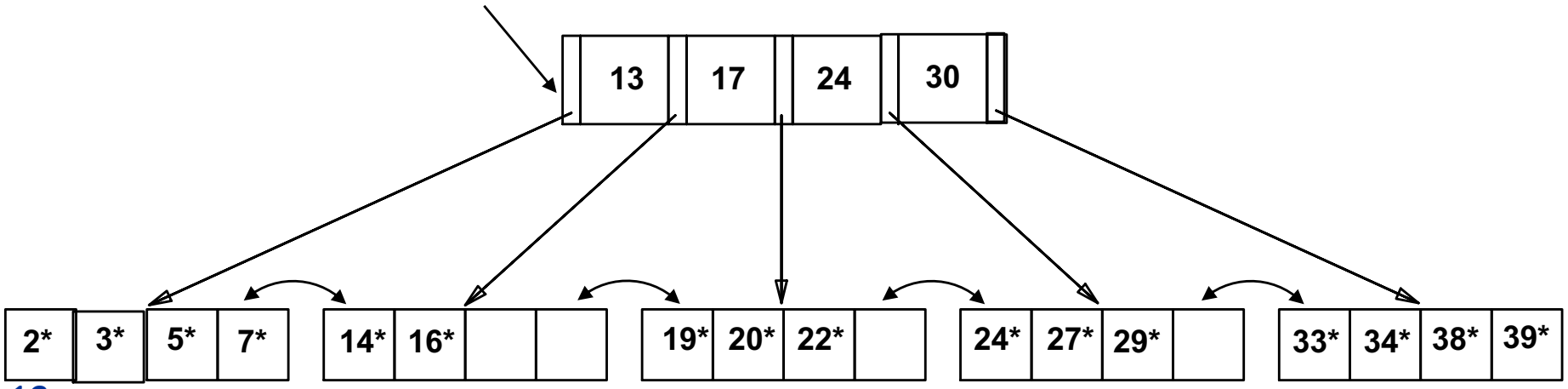
B+ Tree: The Most Widely Used Index

- Root and inner nodes have auxiliary *index entries*
  - A possible value for the search key
  - A pointer to a child
  - Index entries are sorted by search key value within page and across pages of same level



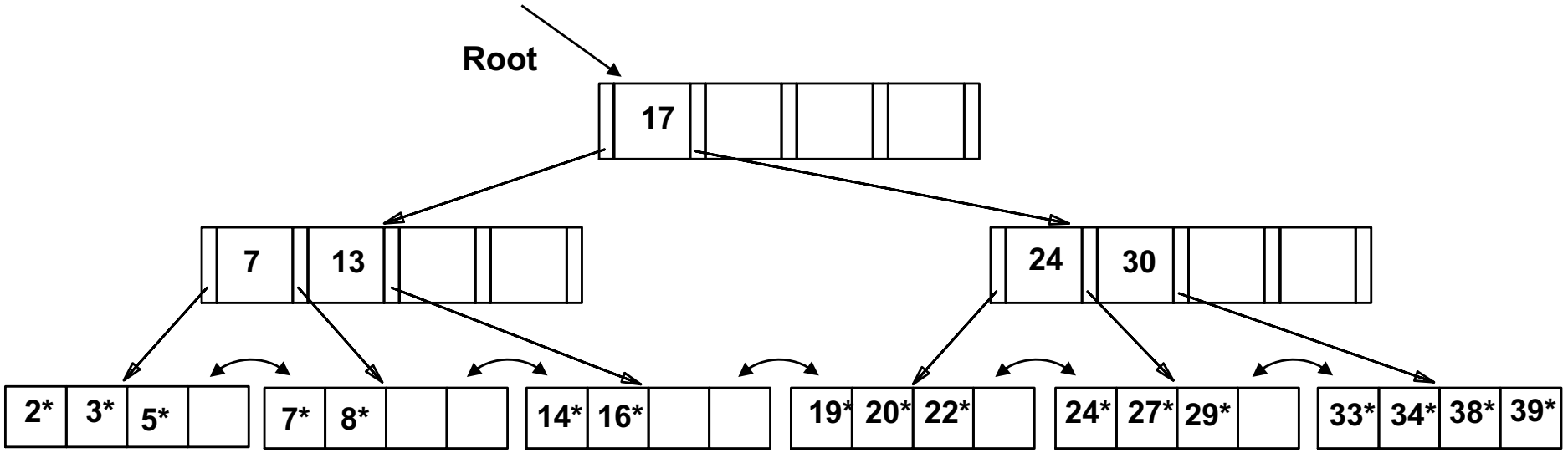
Using an B+tree

- `Select * from Students where sid = 5`
- `Select * from Students where sid = 25`
- Look through root, do key comparison and find the right child (which is a leaf in this example)
  - If tuple exist, get the data page and access tuple via rid
  - If tuple does not exist, immediately return "tuple not found"



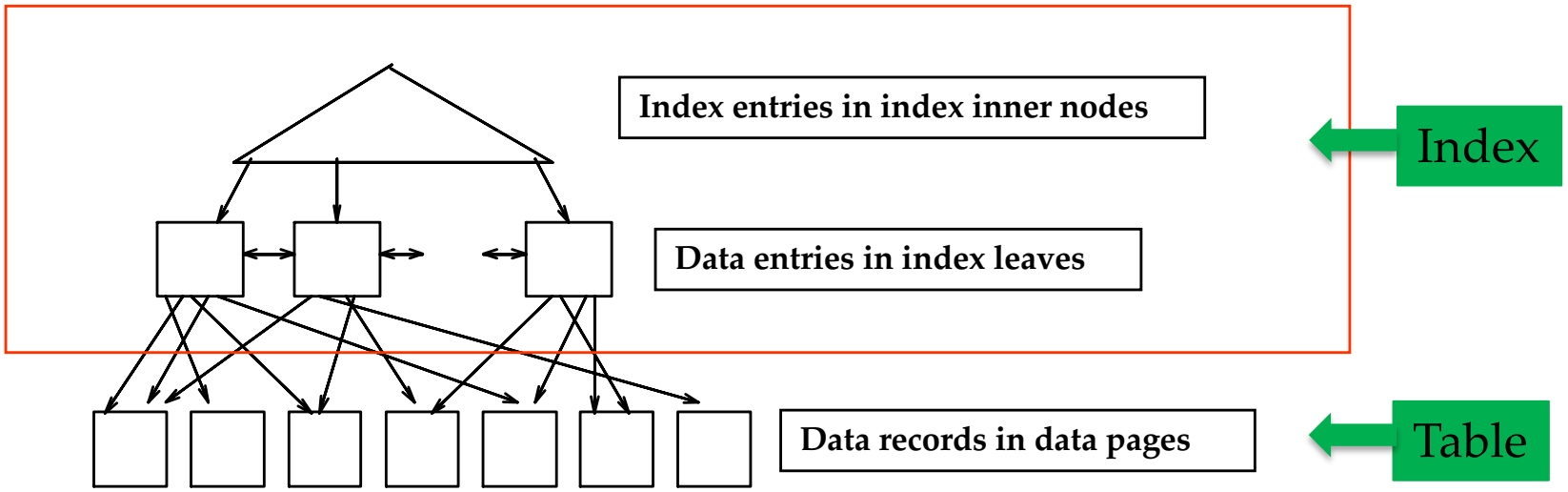
B+ Tree with Multiple Levels

- A hierarchy of nodes
  - Inner nodes with index entries (plus one extra pointer)
    - Root: covers entire range
    - Intermediate nodes: cover sub-ranges
  - Leaf nodes with data entries
- Disk-based: one node = one page



B+ Tree (contd.)

- *height-balanced*.
  - ☆ Each path from root to tree has the same height
  - ☆ Height: number of edges from root to leaf
- F = fanout = number of children for each inner node (~ number of index entries stored in node)
- N = # leaf pages
- Insert/delete at $\log_F N$ cost;
- Minimum 50% occupancy (except for root).
  - Number of index/leaf entries at least 50% of maximum possible number

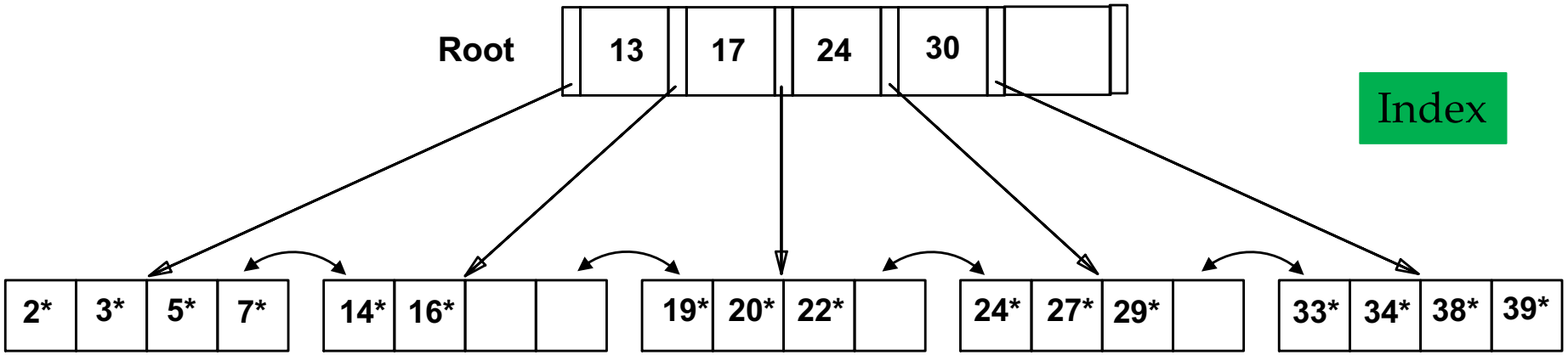


Disclaimer

- Size:
  - Example relation has only 16 tuples
    - You don't build an index for such a relation
  - Think of thousands and millions and more tuples
- Index pages:
  - Inner pages contain hundreds of index entries
  - Leaf pages contain hundreds of data entries

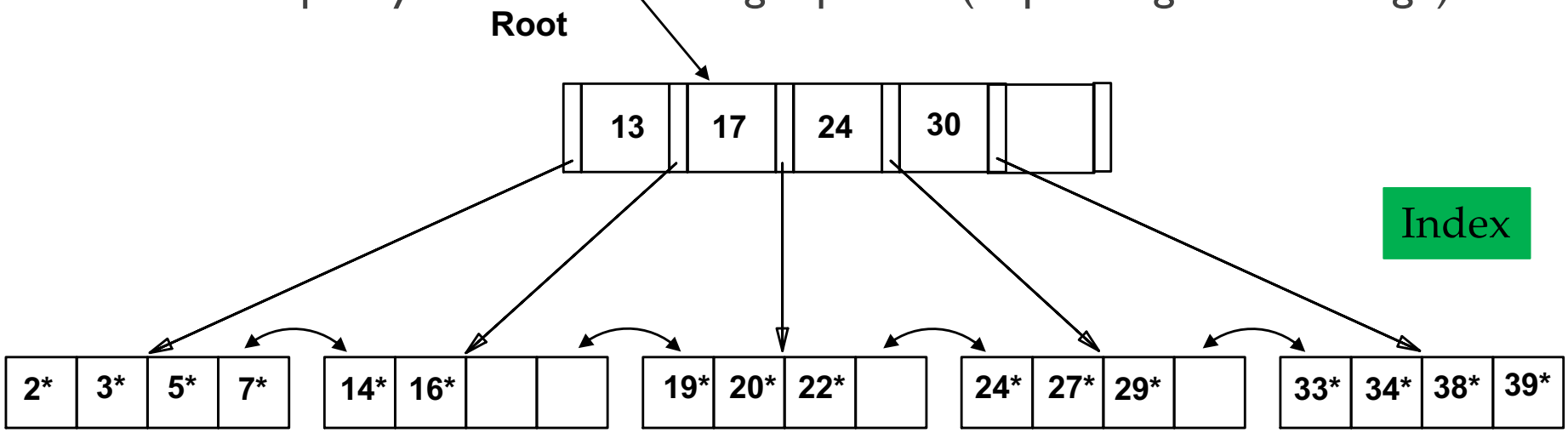
Using a B+ Tree

- `Select * from Students where sid = 5`
- `Select * from Students where sid = 25`
- Look through root, do key comparison and find the right child (which is a leaf in this example)
  - If tuple exist, get the data page and access tuple via rid
  - If tuple does not exist, immediately return "tuple not found"
- Number of pages accessed: three: root, leaf, data page with the record
- Number of I/O:
  - depends of how much of tree is already in the buffer in main memory
  - rough assumption: root and intermediate nodes are always in main memory; index leaves and data pages not in main memory upon first access



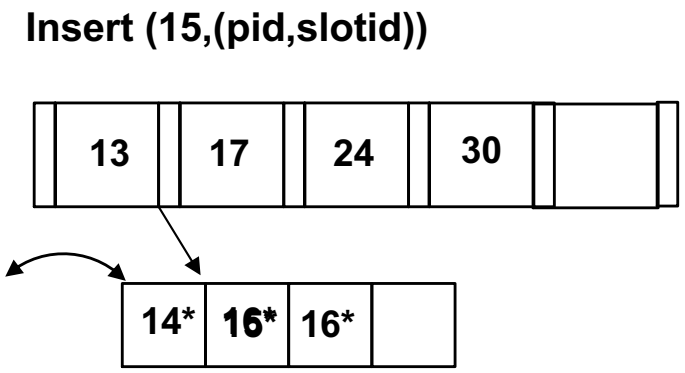
Using B+ Tree

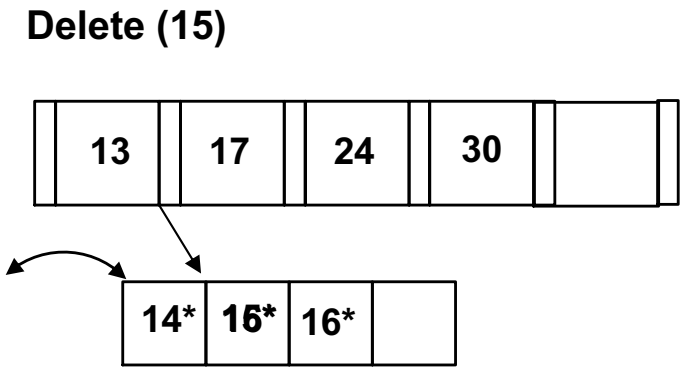
- `Select * from Students where sid = 5`
  - I/O costs:
    - one for leaf page with data entry, one for data page with data record
- `Select * from Students where sid >= 20 and sid < 29`
  - I/O costs:
    - two for leaf pages
    - four for data pages with records
- Good for equality search AND range queries (depending on the range)



Inserting/Deleting a Data Entry

- Find correct leaf *L*.
- Put data entry into *L* / delete data entry from L



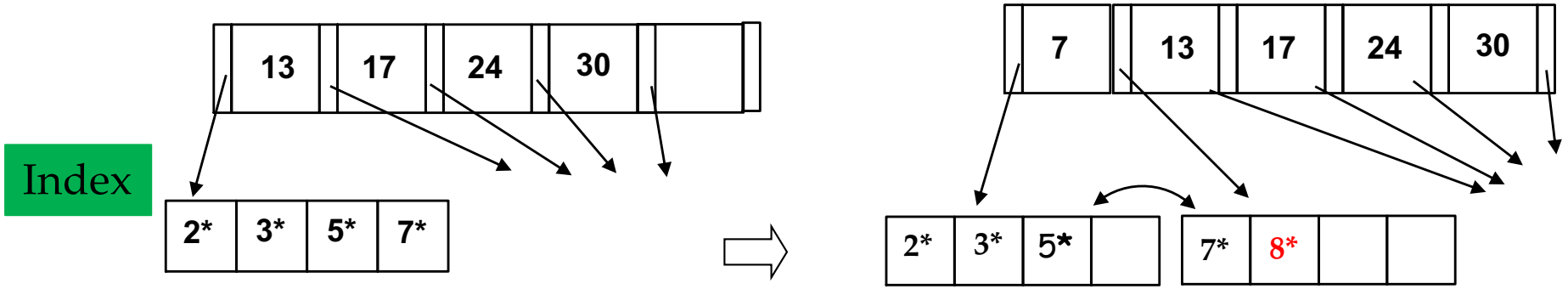


Inserting a Data Entry when leaf page is full

- Find correct leaf *L*.
- Put data entry into *L*.
  - If *L* has enough space, *done*!
  - Else, must *split* *L (into L and a new node L2)*
    - Redistribute entries evenly, **copy up** middle key.
    - Insert index entry pointing to *L2* into parent of *L*.

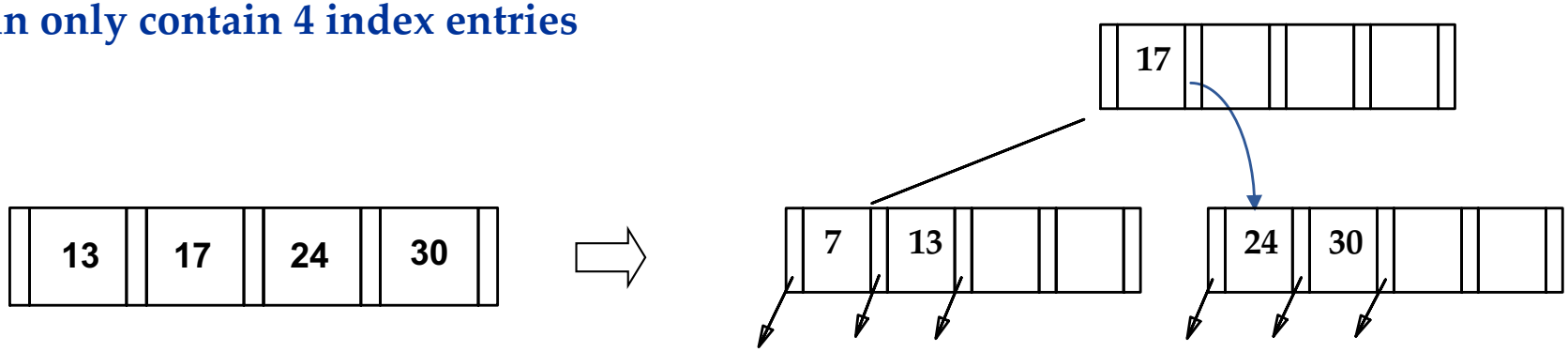
Inserting 8* into Example B+ Tree

Insert into Leaf with leaf split

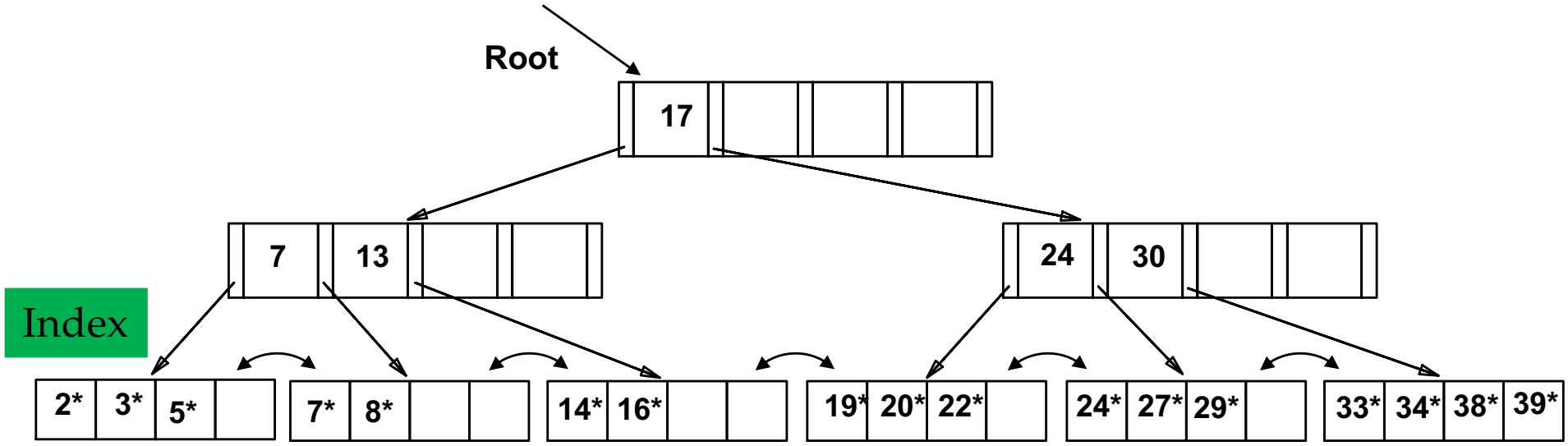


Insert into internal node with node split

Assume that inner pages can only contain 4 index entries



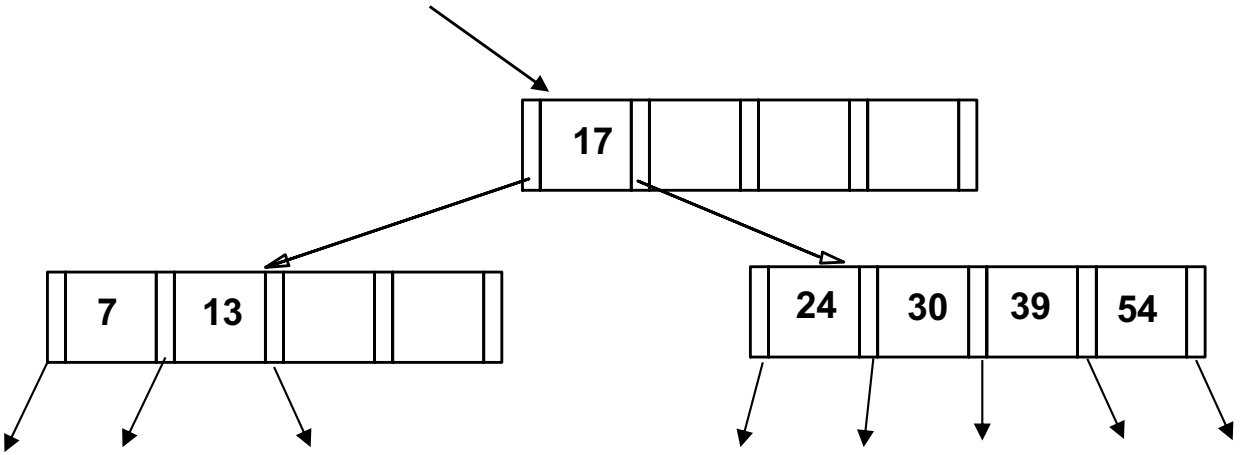
Example: After Inserting 8*



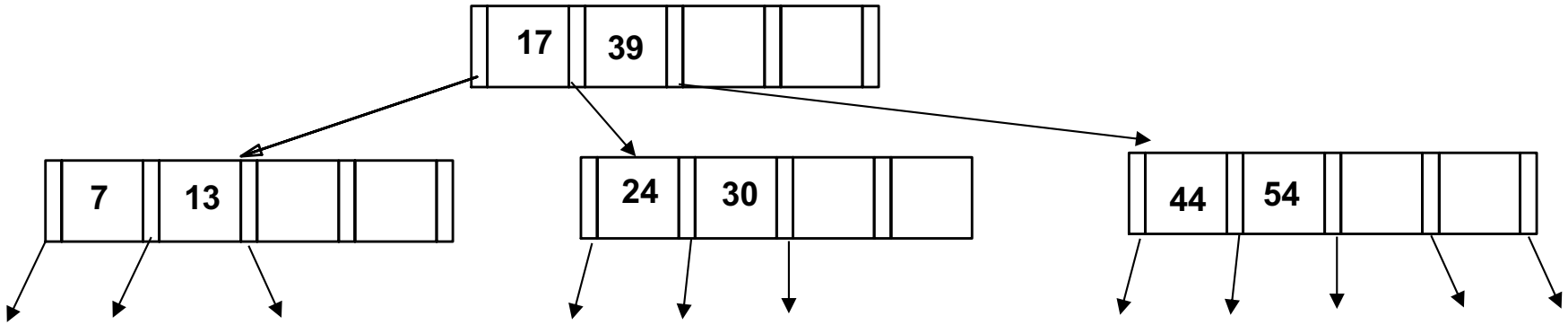
❖ Notice that root was split, leading to increase in height.

❖ In this example, we can avoid split by redistributing entries; however, this is usually not done in practice.

Root Split is rare; usually just one more inner node



Leaf pages not shown



Inserting a Data Entry when leaf page is full

- Find correct leaf *L*.
- Put data entry into *L*.
  - If *L* has enough space, *done*!
  - Else, must *split* *L (into L and a new node L2)*
    - Redistribute entries evenly, **copy up** middle key.
    - Insert index entry pointing to *L2* into parent of *L*.
- This can happen recursively
  - To split index node, redistribute entries evenly, but **push up** middle key. (Contrast with leaf splits.)
- Splits "grow" tree; root split increases height.
  - Tree growth: gets *wider* or *one level taller at top*.

Data Entry Alternatives: indirect Indexing

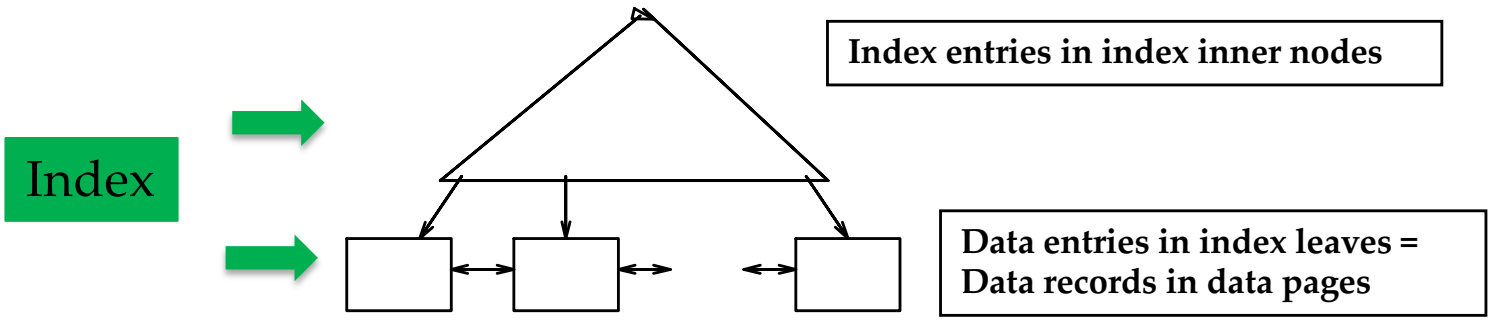
- Indirect Indexing I
  - so far: k* = <**k**, rid of data record with search key value **k**> (indirect indexing)
  - on non-primary key search key: (2022, rid1), (2022, rid2), (2022, rid3), …
    - several entries with the same search key side by side
- Indirect indexing II
  - <**k**, list of rids of data records with search key **k**> (indirect indexing)
  - on non-primary key search key: (2022, (rid1, rid2, rid3,…)), (2023, (rid…
- Comparison:
  - first requires more space (search key repeated)
  - second has variable length data entries
  - second can have large data entries that span a page

Direct Indexing

- Instead of data-entries in index leaves containing rids, they could contain the entire tuple
  - ☆data-entry = tuple
  - ☆no extra data pages
- This is kind of a sorted file with an index on top

Data Entry Alternatives: Direct Index

- Structure
  - <((**k**), full record>
  - Leaf pages contain entire records
    - Data entries = records (not only pointers to them)
    - e.g., index on sid of Skaters
    - (1,lilly,10,16), (2,debby,8,10)…
  - Leafs represent sorted file for data records.
  - Inner nodes above leafs provide faster search
  - At most one direct index per relation
    - Relation can only be sorted by one attribute

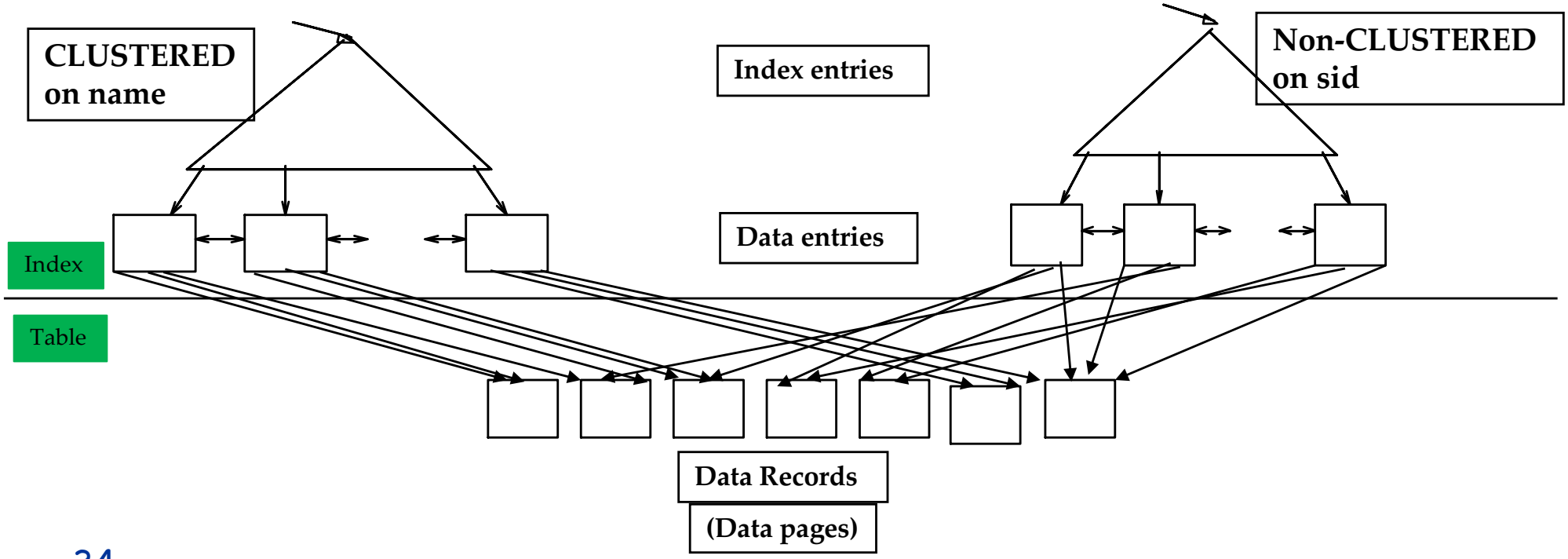


Clustered vs. Non-clustered Index

- Clustered:
  - Relation in file sorted by the search key attributes of the index
- Non-clustered (also referred to as unclustered)
  - Relation in heap file or sorted by an attribute different to the search key attribute of the index.
- A file can be clustered on at most one search key.
- Cost of retrieving data records through index varies *greatly* based on whether index is clustered or not!
- Direct index → clustered index
- Indirect index → can be a clustered index or a non-clustered index

Clustered vs. non-clustered Index

- Same Example – different visualization
  - ☆Indirect clustered index on name
  - ☆Indirect non-clustered on sid



Intuition of cost difference

- Most impact with range queries
- SELECT * from Students where name LIKE 'B%'
- Clustered index
  - Index finds the first data page with a student that starts with B
  - All further students that start with B are on same and adjacent data pages
  - Only small subset of data pages need to be retrieved
- Non-clustered index
  - The students starting with B could be spread across many data pages
  - These data pages are now retrieved in random order when looking them up through index

Primary/Unique Indices

- Primary vs. secondary: If search key contains primary key, then called primary index.
  - *Unique* index: Search key is primary key or unique attribute.
- Secondary index: search key is not unique

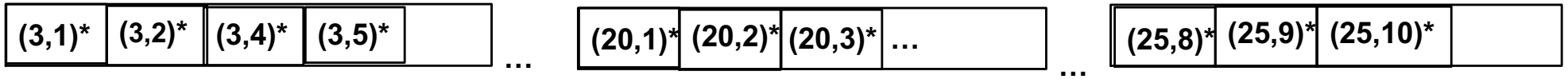
Disclaimer

- Terms
  - Clustered / non-clustered
  - Primary / secondary
  - Direct / indirect

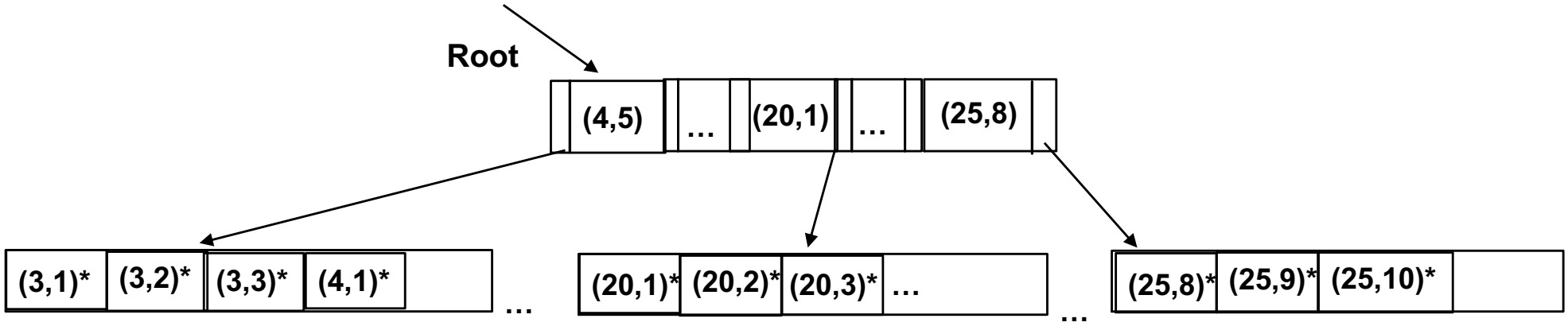
- Are used differently across text books
- Are used differently by different DBMS
- Are used different by different web sources
- So be careful to always understand the definitions of a term!!
- Terms here taken from textbook

Multi-attribute index

- Index on **`Skaters`**`(age,rating);`
- Order is important:
  - Here data entries are first ordered by age
  - Skaters with the same age are then ordered by rating
  - assume youngest skater is 3, oldest is 825 (list of rids: *):
    - the leaf pages then would look like:



What does it support



- What does it support?
  - `SELECT * FROM Skaters WHERE age = 20 AND rating = 5;`
    - Yes
  - `SELECT * FROM Skaters WHERE age = 20 AND rating < 5;`
    - Yes
  - `SELECT * FROM Skaters WHERE age = 20;`
    - yes
  - `SELECT * FROM Skaters WHERE rating < 5;`
    - No
  - `SELECT * FROM Skaters WHERE age = 20 OR rating <=5;`
    - No

Index in DB2

- Simple
  - `CREATE INDEX ind1 ON Students(startyear);`
  - `DROP INDEX ind1;`
- Clustered index
  - `CREATE INDEX ind2 on Students(name) CLUSTER`
- Index also good for referential integrity (uniqueness)
  - `CREATE UNIQUE INDEX indemail ON Students(email)`
- Multi-attribute index
  - `CREATE INDEX ind3 on Students(name,startyear)`
- Additional attributes
  - `CREATE UNIQUE INDEX ind1 ON Students(sid) INCLUDE(name)`
  - Index only on sid
  - Index entries in inner pages only contain search key attribute sid
    - only conditions on index attribute are supported
  - *Data entries* in the leaf pages *additionally* contain name;
    - key value (sid) + name + rid
  - Why?
    - `SELECT name FROM Students WHERE sid = 100`
    - Can be answered without accessing the real data pages of Students relation!